

Color Clash Test Report

Project: Color Clash (v1.0.0)

Test Responsibles

- Mehmet Umur OZU - Developer and Tester
- Mehmet Yasin TOSUN - Developer and Tester
- Mustafa GOZUTOK - Developer and Tester
- Codex Automation Agent - Automated test execution and report preparation

Test Configuration

os	Windows (PowerShell)
node_version	v24.13.0
npm_version	11.6.2
application_stack	TypeScript + Vite web game
build_command	npm.cmd run build
automated_test_command	npm.cmd run test:smoke
manual_test_requirement	Interactive browser session (Chrome or Edge latest)

Execution Summary

Total test cases: 16

PASS	16
------	----

Test Inputs and Test Results

TC-BUILD-001 - Production build smoke check [PASS]

Use Case: UC-08

Type: Automated

Priority: High

Preconditions: Dependencies installed

Test Inputs: Build command execution

Expected Result: Build completes with no error

Actual Result: Build completed successfully. Vite transformed 30 modules and produced dist/index.html and JS bundle.

Evidence: Command output includes: "vite v6.4.1 building for production" and "built in 310ms".

Steps:

1. Run npm.cmd run build
2. Observe TypeScript and Vite build output

TC-AUTO-001 - Character roster consistency [PASS]

Use Case: UC-01

Type: Automated

Priority: High

Preconditions: Temporary CJS transpilation available

Test Inputs: No user input; direct factory call

Expected Result: Roster contains Fighter, Geomancer, Eclipse and correct mode pairs

Actual Result: Roster length and type/mode mapping matched expected values.

Evidence: Node test: Character roster exposes expected archetypes and mode pairs

Steps:

3. Load CharacterFactory
4. Call getCharacterRoster()

TC-AUTO-002 - Match player initialization [PASS]

Use Case: UC-01

Type: Automated

Priority: High

Preconditions: Temporary CJS transpilation available

Test Inputs: Character types: Fighter, Geomancer

Expected Result: Player IDs and spawn coordinates are correct

Actual Result: IDs, positions and initial modes were as expected.

Evidence: Node test: CharacterFactory creates match players with expected ids and spawn coordinates

Steps:

5. Create match players with Fighter and Geomancer
6. Verify ids, spawn positions and default modes

TC-AUTO-003 - Mode switch cycle [PASS]

Use Case: UC-03

Type: Automated

Priority: High

Preconditions: Temporary CJS transpilation available

Test Inputs: Switch command sequence

Expected Result: Mode cycles Fire -> Water -> Fire

Actual Result: Mode cycle behavior matched expected sequence.

Evidence: Node test: SwitchModeCommand cycles character mode and returns to initial mode

Steps:

7. Execute SwitchModeCommand twice on Fighter

TC-AUTO-004 - Movement command flags [PASS]

Use Case: UC-02

Type: Automated

Priority: Medium

Preconditions: Temporary CJS transpilation available

Test Inputs: left key press and release simulation

Expected Result: Input flags and moving state update correctly

Actual Result: Flags toggled correctly and moving state reflected press/release.

Evidence: Node test: MoveCommand updates movement flags and moving state

Steps:

8. Execute MoveCommand(left,true)
9. Execute MoveCommand(left,false)

TC-AUTO-005 - Attack hit applies damage [PASS]

Use Case: UC-02

Type: Automated

Priority: High

Preconditions: Temporary CJS transpilation available

Test Inputs: Attack command with overlap position setup

Expected Result: Defender health decreases

Actual Result: Defender health dropped below initial value.

Evidence: Node test: Attack state applies damage when opponent is inside hitbox

Steps:

10. Place players in hitbox range
11. Trigger attack and advance state updates

TC-AUTO-006 - Dark absorb to Light bonus conversion [PASS]

Use Case: UC-03

Type: Automated

Priority: High

Preconditions: Temporary CJS transpilation available

Test Inputs: Incoming damage = 30

Expected Result: First Light attack is boosted, second returns to base Light damage

Actual Result: First Light attack exceeded baseline; second returned to baseline.

Evidence: Node test: Dark mode absorbed damage is consumed as bonus by next Light attack

Steps:

12. Switch Eclipse to Dark mode
13. Apply incoming damage
14. Recover from hit state and switch to Light
15. Perform two Light attacks

TC-AUTO-007 - Collision midpoint calculation [PASS]

Use Case: UC-02

Type: Automated

Priority: Medium

Preconditions: Temporary CJS transpilation available

Test Inputs: Rect A/B overlap and non-overlap samples

Expected Result: Overlap returns midpoint, non-overlap returns null

Actual Result: Midpoint and null behaviors matched expectation.

Evidence: Node test: CollisionSystem detects overlap and computes overlap midpoint

Steps:

16. Evaluate overlap rectangles

17. Evaluate non-overlap rectangles

TC-AUTO-008 - Character collision push-apart [PASS]

Use Case: UC-02

Type: Automated

Priority: Medium

Preconditions: Temporary CJS transpilation available

Test Inputs: Player x positions: 400 and 430

Expected Result: Post-resolution distance is greater than pre-resolution distance

Actual Result: Distance increased after collision resolution.

Evidence: Node test: CollisionSystem resolveCharacterCollision increases distance between overlapping players

Steps:

18. Create overlapping players

19. Run resolveCharacterCollision()

20. Compare horizontal distance before and after

TC-MAN-001 - Normal mode setup and start flow [PASS]

Use Case: UC-01

Type: Manual

Priority: High

Preconditions: Browser session and running dev server

Test Inputs: Player names and character choices

Expected Result: Match starts and summary reflects selected values

Actual Result: Executed manually. Match started correctly and setup summary reflected selected names and archetypes.

Evidence: Manual browser execution by project tester

Steps:

21. Choose normal mode
22. Set player names and archetypes
23. Press Start Match

TC-MAN-002 - Keyboard controls in active match [PASS]

Use Case: UC-02

Type: Manual

Priority: High

Preconditions: Browser session with active match

Test Inputs: Gameplay keyboard actions

Expected Result: Movement, attack and mode switch are responsive for both players

Actual Result: Executed manually. Player 1 and Player 2 controls responded as expected (movement, attack, mode switch).

Evidence: Manual browser execution by project tester

Steps:

24. Use WASD/F/G for Player 1
25. Use Arrow/K/L for Player 2

TC-MAN-003 - Winner detection and auto restart [PASS]

Use Case: UC-04

Type: Manual

Priority: High

Preconditions: Browser session with active normal match

Test Inputs: Gameplay actions until KO

Expected Result: Winner announced and match restarts in ~3 seconds

Actual Result: Executed manually. Winner overlay appeared and normal mode auto-restart flow worked as expected.

Evidence: Manual browser execution by project tester

Steps:

- 26. Finish one player health bar
- 27. Observe winner overlay and restart timing

TC-MAN-004 - Tournament setup and round progression [PASS]

Use Case: UC-05

Type: Manual

Priority: High

Preconditions: Browser session and tournament mode selected

Test Inputs: Tournament size 2/4/6/8, participant names and characters

Expected Result: Bracket runs sequentially and round progress overlay is shown

Actual Result: Executed manually. Tournament bracket progressed sequentially and round overlays were shown correctly.

Evidence: Manual browser execution by project tester

Steps:

- 28. Set tournament size and participants
- 29. Start tournament
- 30. Observe sequential matches and round overlay

TC-MAN-005 - Final podium and champion summary [PASS]

Use Case: UC-06

Type: Manual

Priority: High

Preconditions: Browser session after tournament completion

Test Inputs: Final match outcome

Expected Result: 1st/2nd/3rd place rendering and champion subtitle are visible

Actual Result: Executed manually. Final podium and champion subtitle were displayed correctly.

Evidence: Manual browser execution by project tester

Steps:

- 31. Complete tournament final
- 32. Observe final overlay and result list

TC-MAN-006 - Global shortcuts (P, R, Esc) [PASS]

Use Case: UC-07

Type: Manual

Priority: Medium

Preconditions: Browser session with active gameplay

Test Inputs: Shortcut key presses

Expected Result: Pause toggles, restart resets, Esc returns setup overlay

Actual Result: Executed manually. P, R and Esc shortcuts behaved as expected.

Evidence: Manual browser execution by project tester

Steps:

- 33. Press P to pause/resume
- 34. Press R to restart
- 35. Press Esc to return setup

TC-MAN-007 - Input guard while editing fields [PASS]

Use Case: UC-07

Type: Manual

Priority: Medium

Preconditions: Browser session with cursor in input/select

Test Inputs: Typing and gameplay keys while field is focused

Expected Result: Gameplay commands are ignored while editing inputs

Actual Result: Executed manually. Gameplay commands were ignored while editing focused form inputs.

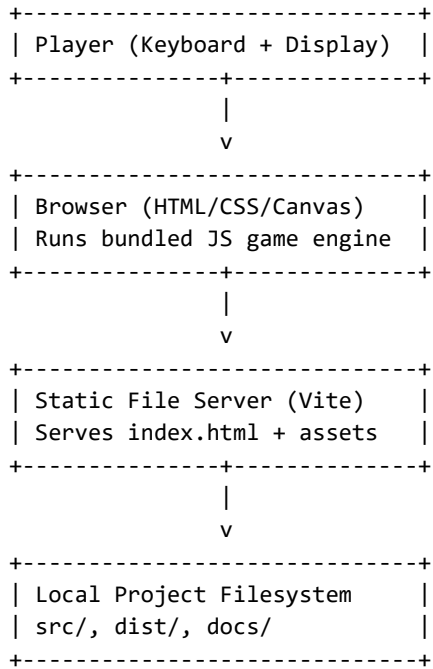
Evidence: Manual browser execution by project tester

Steps:

36. Focus name input or select field
37. Press gameplay keys

Related Deployment Diagram

Local Deployment Diagram



Note: Current project deployment is client-side and local.